



UNIVERSITY OF GENEVA

Faculty of Science

Exercise #5

Introduction to Computational Finance
14X030

Michel Jean Joseph Donnet



Contents

1 Limit Order Book — Maslov model	3
2 Heterogeneous Order Sizes and Trader Behavior on LOB Dynamics	16



1 Limit Order Book — Maslov model

The goal of this series is to implement the Maslov model (check the paper for more details), which allows to reproduce interesting stylized facts while being relatively simple.

We consider the following version of the model:

- One trader at each time step.
 - Buyer or seller, with probability $1 - q$ and q .
 - Issues a market order (buy at ask / sell at bid) with probability $1 - r$ or a limit order with probability r . The price for the limit order is defined as $p - K$ when buying and $p + K$ when selling, with p the market price, defined as the last price said in the market. K is a positive random variable.
 - Each trader buys or sells a quantity of one.
 - An order cannot be canceled.
 - If an order cannot be executed (e.g. a sell market order but there is no buy order in the LOB), then nothing happens at this time step.
1. Implement the procedure above and unroll it for 1000 iterations. Use $q = r = 0.5$ and $K = 1$ with probability 1. Plot the time series of the market, ask, bid and mid prices $p(t)$, $a(t)$, $b(t)$ and $m(t) = \frac{b(t)+a(t)}{2}$. Are there any difference between the mid and the market prices?

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(2026)

def Maslov(iterations: int = 1000, q: float = 0.5, r: float = 0.5, K: float = 1):
    # Initial price
    p = [100.0]
    orders = []
    sells = []
    best_asks = []
    best_bids = []
    for i in range(iterations):
        # Buy process
        if np.random.rand() <= 1 - q:
            # Buyer
            if np.random.rand() <= 1 - r:
                if sells:
                    sells.sort()
                    p.append(sells.pop())
            else:
                p.append(p[-1])
        # Market maker
        else:
            orders.append(p[-1] - K)
```



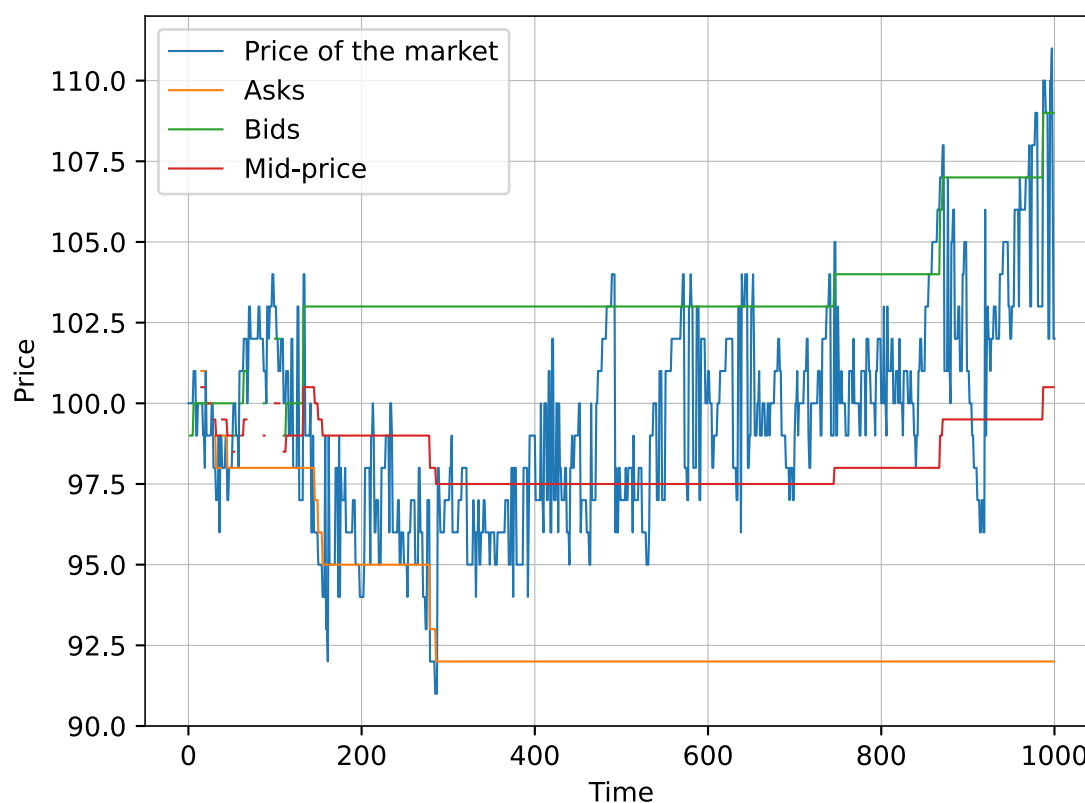
```

        p.append(p[-1])
    # Sell process
    else:
        # Seller
        if np.random.rand() <= 1 - r:
            if orders:
                orders.sort(reverse=True)
                p.append(orders.pop())
            else:
                p.append(p[-1])
        # Market maker
        else:
            sells.append(p[-1] + K)
            p.append(p[-1])
    if sells:
        best_asks.append(np.min(sells))
    else:
        best_asks.append(np.nan)
    if orders:
        best_bids.append(np.max(orders))
    else:
        best_bids.append(np.nan)
    p = np.array(p)
    best_asks = np.array(best_asks)
    best_bids = np.array(best_bids)
    mid_price = (best_asks + best_bids) / 2
    spread = best_asks - best_bids
    return p, best_asks, best_bids, mid_price, spread, orders, sells

p, a, b, mid_price, spread, orders, sells = Maslov()

plt.figure()
plt.plot(p, label="Price of the market")
plt.plot(a, label="Asks")
plt.plot(b, label="Bids")
plt.plot(mid_price, label="Mid-price")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()

```



2. Plot the time series of returns of the market price, what do you observe?

```
def returns(ts: np.ndarray, N: int = 1) -> np.ndarray:
    result = ts.copy()
    result[:N] = np.nan
    return (result[N:] - result[:-N]) / result[:-N]

plt.figure()
plt.plot(returns(p))
plt.title("Time series of returns of the market price")
plt.xlabel("Time")
plt.ylabel("Returns")
plt.show()
```

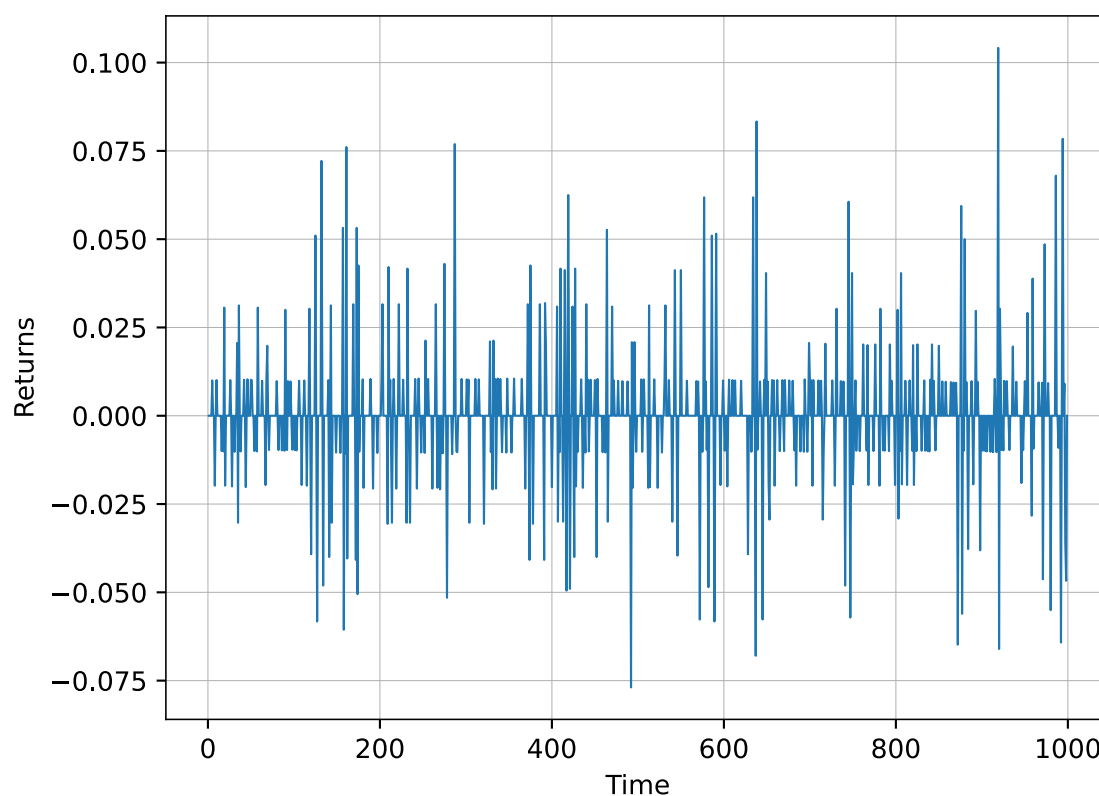


Figure 1: Time series of returns of the market price

3. Plot the histogram of the values of this series. What can you say of this distribution? Is it a normal distribution?

```
plt.figure()
plt.hist(returns(p), bins=25)
plt.title("Histogram of Returns")
plt.xlabel("Returns")
plt.ylabel("Frequency")
plt.show()
```

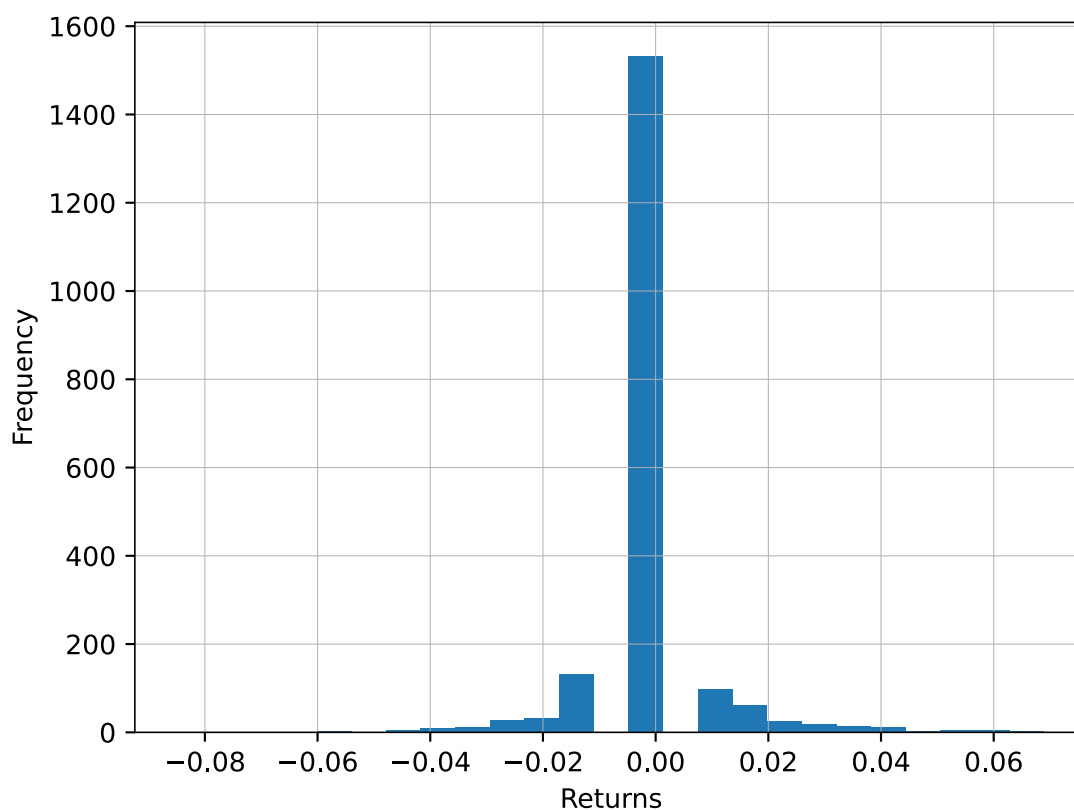


Figure 2: Histogram of Returns

4. Plot and comment the ACF graph of this series.

```
from statsmodels.graphics.tsaplots import plot_acf

plot_acf(returns(p), lags=np.arange(51))
plt.title("ACF of price evolution")
plt.xlabel("Lags")
plt.ylabel("Autocorrelation")
plt.show()
```

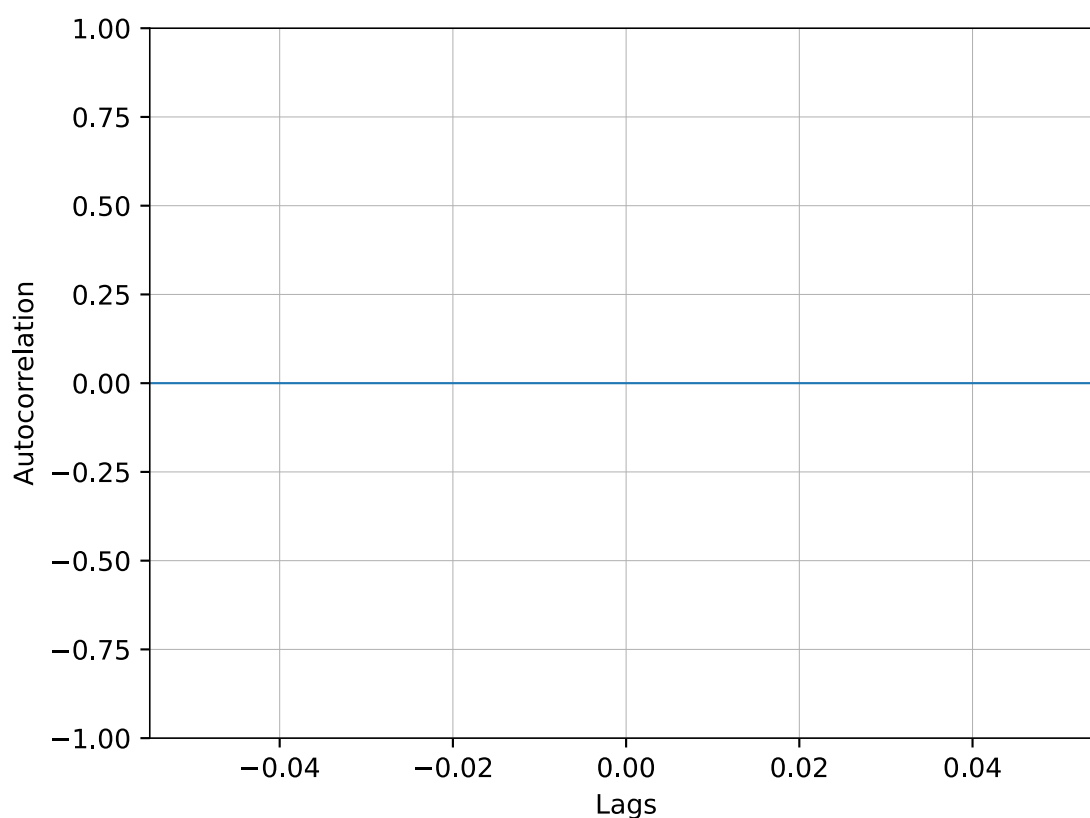


Figure 3: ACF of price evolution

5. Compute the bid-ask spread $s(t) = a(t) - b(t)$ and plot its time series.

```
plt.figure()
plt.plot(spread)
plt.title("Time series of the bid-ask spread")
plt.xlabel("Time")
plt.ylabel("Spread")
plt.show()
```

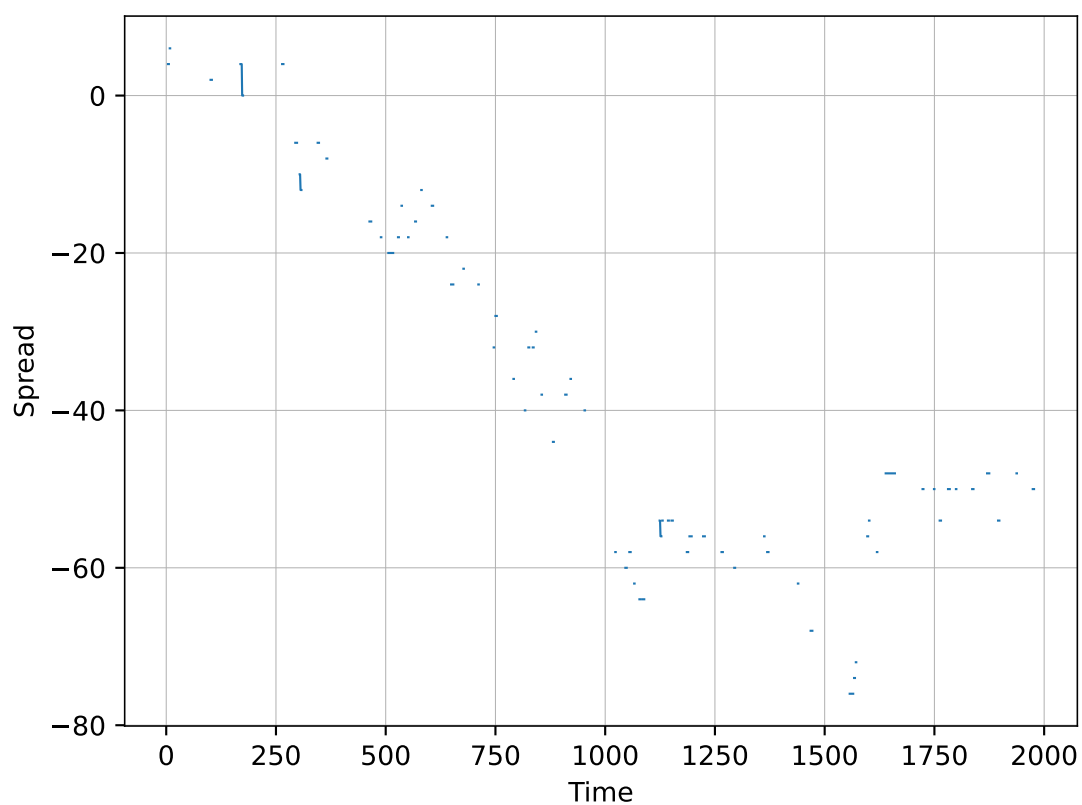


Figure 4: Time series of the bid-ask spread

6. Plot a histogram of the spread values. Comment on whether the distribution is fat-tailed.

```
plt.figure()
plt.hist(spread, bins=25)
plt.title("Histogram of the bid-ask spread")
plt.xlabel("Spread")
plt.ylabel("Frequency")
plt.show()
```

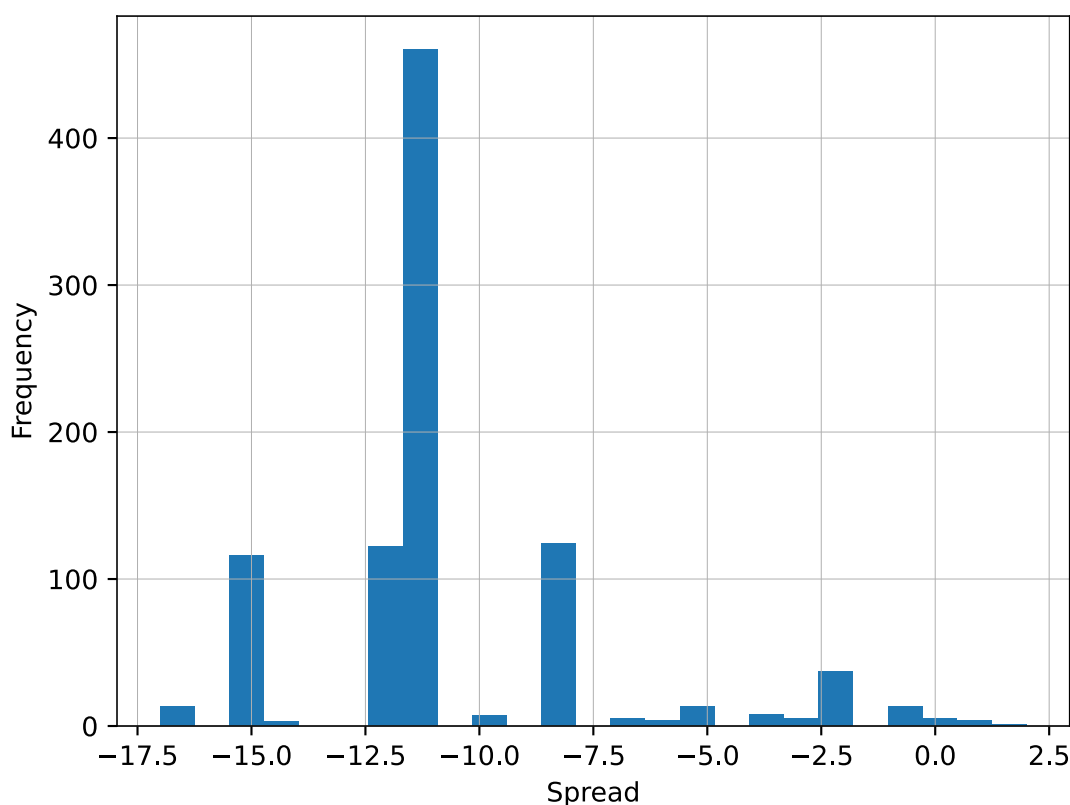


Figure 5: Histogram of the bid-ask spread

- For selected time points, analyze the order book depth: for a range of price levels relative to the best bid/ask, compute the aggregated number of orders.

```
def order_book_depth(sells: list, best_ask: float, levels: int = 5):
    depth = []
    for level in range(1, levels + 1):
        price_level = best_ask + level
        depth.append(np.sum(np.array(sells) <= price_level))
    return np.array(depth).astype(int)

def sell_order_book_depth(orders: list, best_bid: float, levels: int = 5):
    depth = []
    for level in range(1, levels + 1):
        price_level = best_bid - level
        depth.append(np.sum(np.array(orders) >= price_level))
    return np.array(depth).astype(int)

# Example of computing order book and sell book depth at the last time point
buy_order_depth = order_book_depth(sells, a[-1])
sell_order_depth = sell_order_book_depth(orders, b[-1])
print("Order book depth (asks):", buy_order_depth)
print("Order book depth (bids):", sell_order_depth)
```



```
Order book depth (asks): [ 4  4  9 15 21]
Order book depth (bids): [ 2  4  5 10 18]
```

8. Plot the depth distribution and discuss its shape in relation to empirical observations.

```
width = 0.35
plt.figure()
plt.bar(
    np.arange(1, len(buy_order_depth) + 1) - width / 2,
    buy_order_depth,
    width=width,
    label="Ask Depth",
)
plt.bar(
    np.arange(1, len(sell_order_depth) + 1) + width / 2,
    sell_order_depth,
    width=width,
    label="Bid Depth",
)
plt.title("Order Book Depth Distribution")
plt.xlabel("Price Level")
plt.ylabel("Number of Orders")
plt.grid(False)
plt.legend()
plt.show()
```

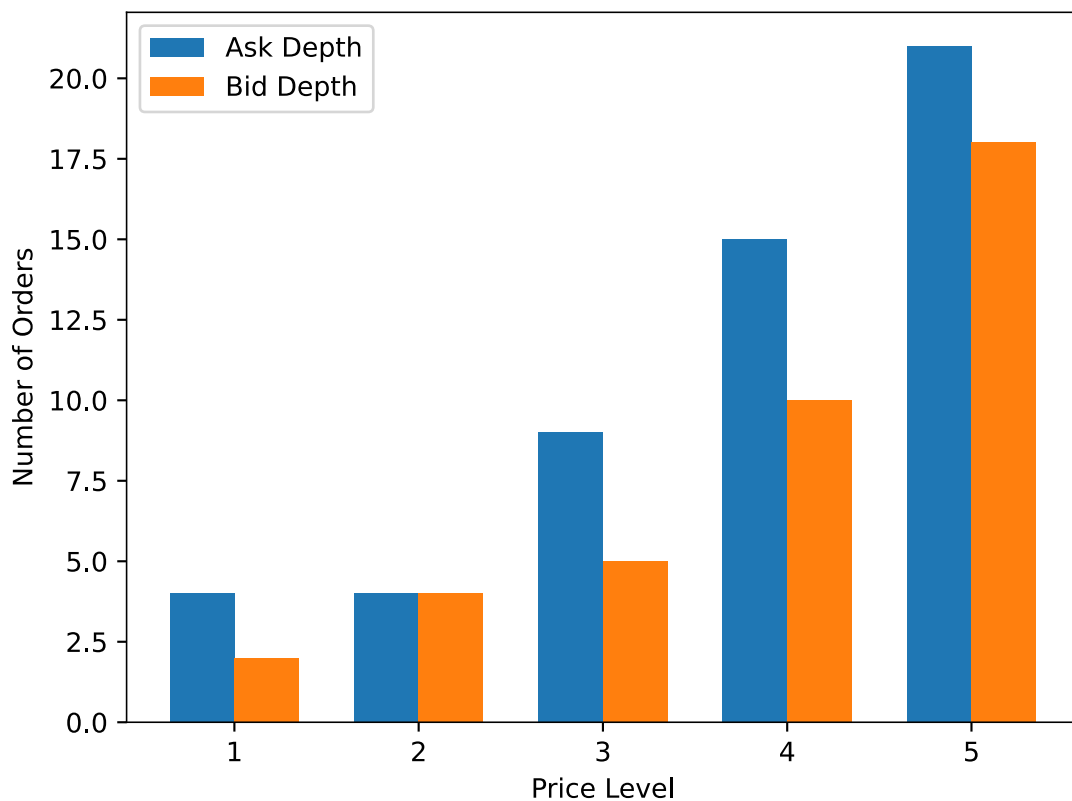
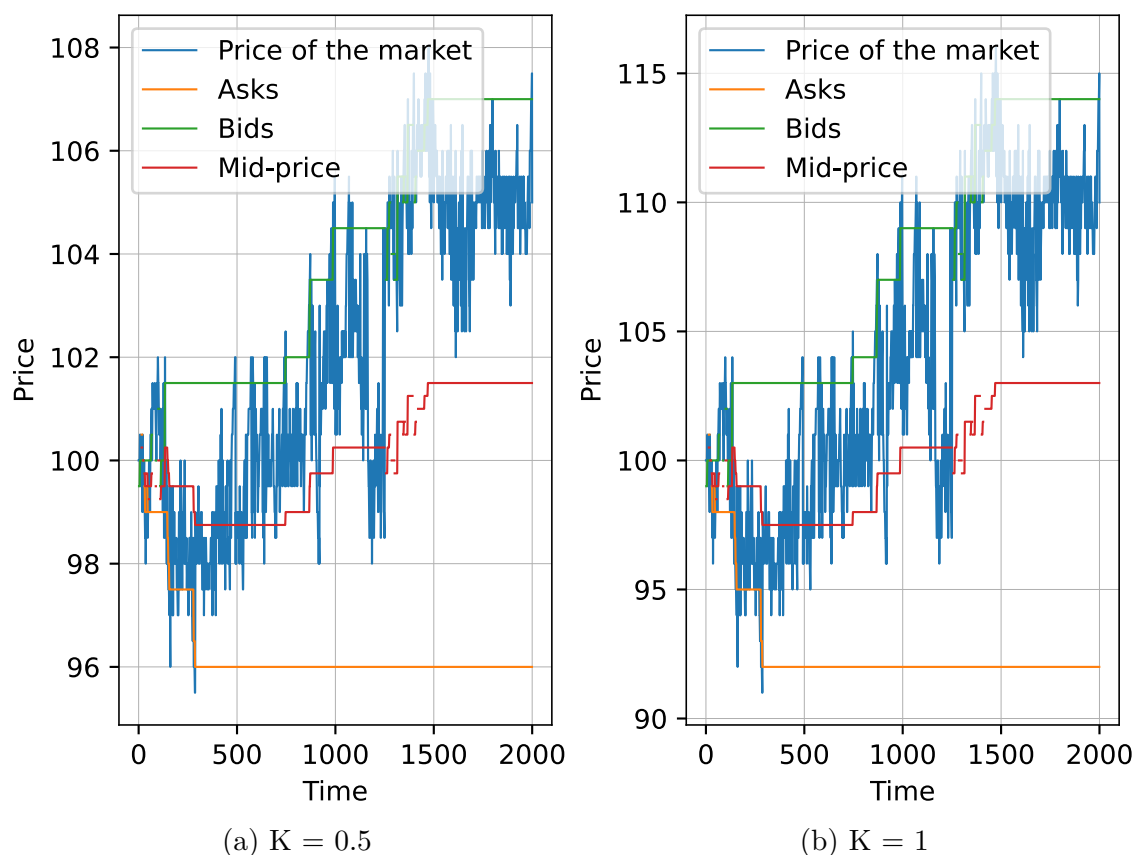


Figure 6: Order Book Depth Distribution



9. Change the parameters of the model to understand how they influence the market and comment about this point.

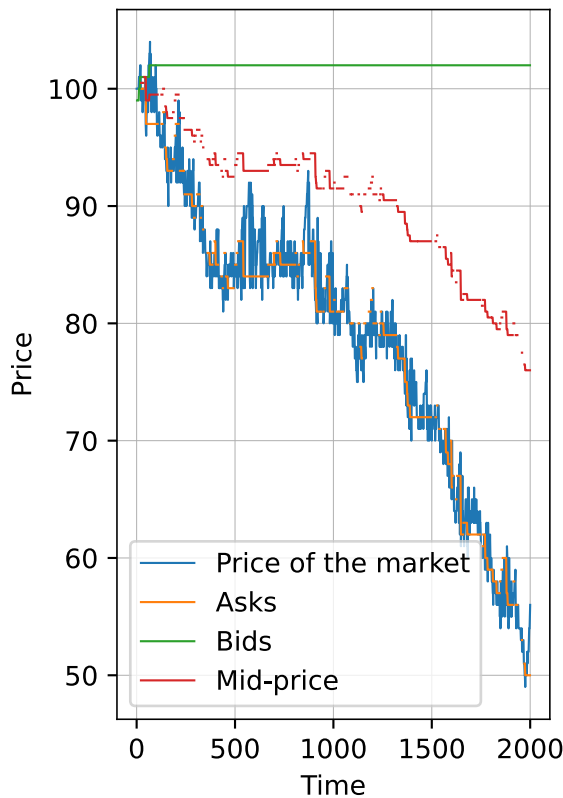
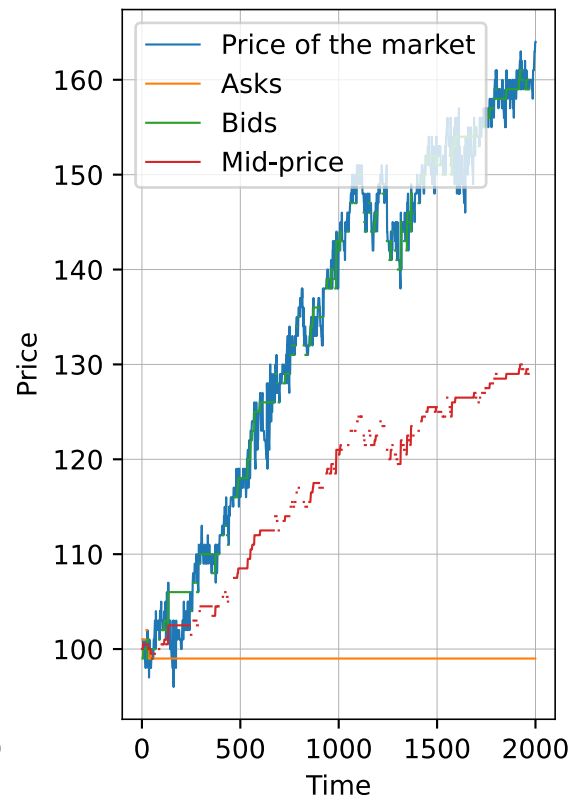
```
# Example of changing parameter K
plt.figure()
plt.suptitle("Comparison of price evolution with different K values")
plt.subplot(1, 2, 1)
plt.title("K = 0.5")
np.random.seed(2026)
p, a, b, mid_price, spread, orders, sells = Maslov(iterations=2000, q=0.5,
r=0.5, K=0.5)
plt.plot(p, label="Price of the market")
plt.plot(a, label="Asks")
plt.plot(b, label="Bids")
plt.plot(mid_price, label="Mid-price")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
np.random.seed(2026)
p, a, b, mid_price, spread, orders, sells = Maslov(iterations=2000, q=0.5,
r=0.5, K=1)
plt.subplot(1, 2, 2)
plt.title("K = 1")
plt.plot(p, label="Price of the market")
plt.plot(a, label="Asks")
plt.plot(b, label="Bids")
plt.plot(mid_price, label="Mid-price")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Figure 7: Comparison of price evolution with different K values

```
# Example of changing parameter q
plt.figure()
plt.suptitle("Comparison of price evolution with different q values")
plt.subplot(1, 2, 1)
plt.title("q = 0.4")
np.random.seed(2026)
p, a, b, mid_price, spread, orders, sells = Maslov(iterations=2000, q=0.4,
r=0.5, K=1)
plt.plot(p, label="Price of the market")
plt.plot(a, label="Asks")
plt.plot(b, label="Bids")
plt.plot(mid_price, label="Mid-price")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
np.random.seed(2026)
p, a, b, mid_price, spread, orders, sells = Maslov(iterations=2000, q=0.6,
r=0.5, K=1)
plt.subplot(1, 2, 2)
plt.title("q = 0.6")
plt.plot(p, label="Price of the market")
plt.plot(a, label="Asks")
plt.plot(b, label="Bids")
plt.plot(mid_price, label="Mid-price")
```



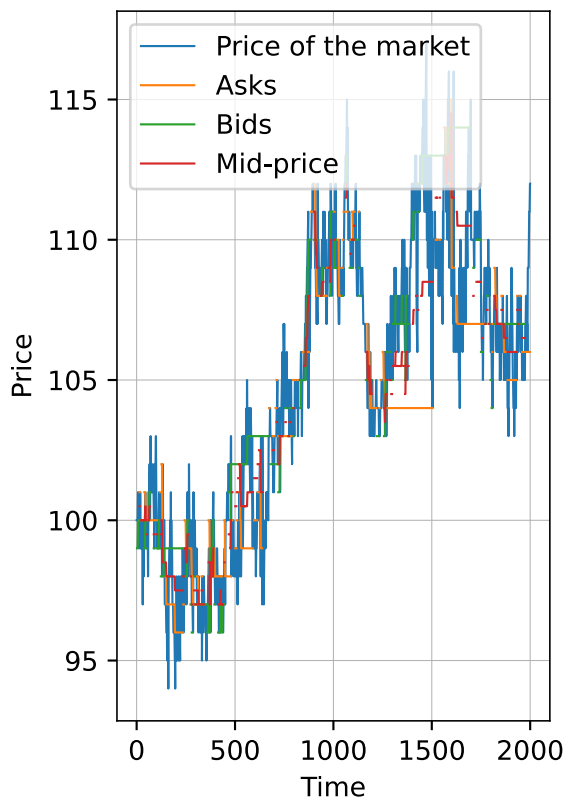
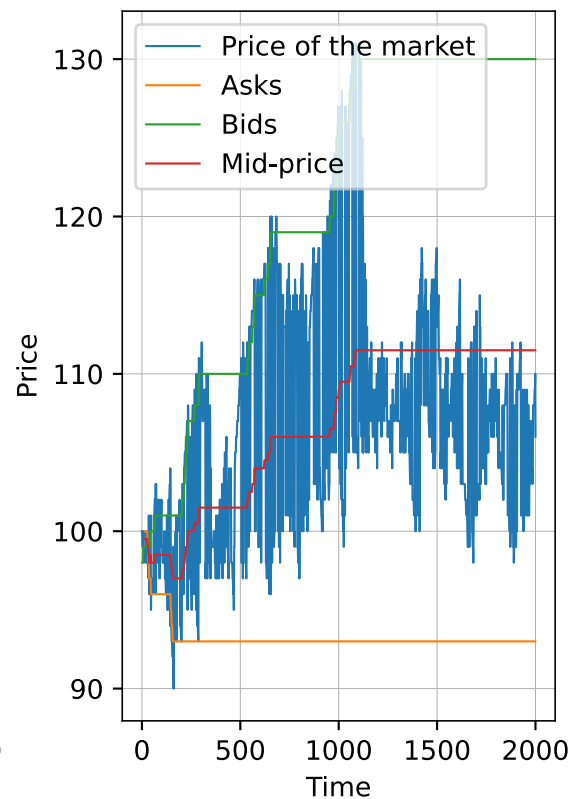
```
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()
```

(a) $q = 0.4$ (b) $q = 0.6$ Figure 8: Comparison of price evolution with different q values

```
# Example of changing parameter r
plt.figure()
plt.suptitle("Comparison of price evolution with different r values")
plt.subplot(1, 2, 1)
plt.title("r = 0.4")
np.random.seed(2026)
p, a, b, mid_price, spread, orders, sells = Maslov(iterations=2000, q=0.5,
r=0.4, K=1)
plt.plot(p, label="Price of the market")
plt.plot(a, label="Asks")
plt.plot(b, label="Bids")
plt.plot(mid_price, label="Mid-price")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
np.random.seed(2026)
p, a, b, mid_price, spread, orders, sells = Maslov(iterations=2000, q=0.5,
r=0.6, K=1)
```



```
plt.subplot(1, 2, 2)
plt.title("r = 0.6")
plt.plot(p, label="Price of the market")
plt.plot(a, label="Asks")
plt.plot(b, label="Bids")
plt.plot(mid_price, label="Mid-price")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()
```

(a) $r = 0.4$ (b) $r = 0.6$ Figure 9: Comparison of price evolution with different r values



2 Heterogeneous Order Sizes and Trader Behavior on LOB Dynamics

Finally, you will extend the basic Maslov model to include heterogeneity in order sizes and trader behavior. You will investigate how these extensions influence the market price dynamics, volatility, and autocorrelation structure of returns.

We now consider the following version of the model:

- Instead of assuming every order has unit size, assume that order sizes are drawn from a distribution (for example, a discrete uniform distribution between 1 and 5).
- Introduce two types of traders:
 - **Aggressive Traders:** With a higher probability (e.g., 0.8) they submit market orders.
 - **Passive Traders:** With a higher probability (e.g., 0.8) they submit limit orders.
- Let each trader, at each time step, be randomly classified as aggressive or passive.

```
def Maslov_Extended(
    iterations: int = 1000,
    q: float = 0.5,
    r: float = 0.5,
    K_range: tuple = (0.5, 1.5),
    aggressive_prob: float = 0.8,
    passive_prob: float = 0.8,
    order_size_range: tuple = (1, 5),
):
    # Initial price
    p = [100.0]
    orders = []
    sells = []
    best_asks = []
    best_bids = []
    for i in range(iterations):
        # Randomly classify trader as aggressive or passive
        aggressive_trader = np.random.rand() < 0.5
        is_seller = np.random.rand() < q
        order_size = np.random.randint(*order_size_range)
        if aggressive_trader:
            market_order = np.random.rand() < aggressive_prob
        else:
            market_order = np.random.rand() < passive_prob

        if market_order:
            if is_seller:
                order_number = min(order_size, len(orders))
                orders.sort(reverse=True)
                # Add best bid price to the price series
                p.append(orders[0] if order_number > 0 else p[-1])
                for _ in range(order_number):
                    orders.pop()
```



```

        else:
            sell_number = min(order_size, len(sells))
            sells.sort()
            # Add best ask price to the price series
            p.append(sells[0] if sell_number > 0 else p[-1])
            for _ in range(sell_number):
                sells.pop()
    else:
        if is_seller:
            for _ in range(order_size):
                sells.append(p[-1] + np.random.uniform(*K_range))
            p.append(p[-1])
        else:
            for _ in range(order_size):
                orders.append(p[-1] - np.random.uniform(*K_range))
            p.append(p[-1])
    best_asks.append(np.min(sells) if sells else np.nan)
    best_bids.append(np.max(orders) if orders else np.nan)
    p = np.array(p)
    best_asks = np.array(best_asks)
    best_bids = np.array(best_bids)
    mid_price = (best_asks + best_bids) / 2
    spread = best_asks - best_bids
    return p, best_asks, best_bids, mid_price, spread, orders, sells

```

1. Run the simulation for 1000 iterations with the extended model.

```

p_ext, a_ext, b_ext, mid_price_ext, spread_ext, orders_ext, sells_ext = (
    Maslov_Extended()
)

plt.figure()
plt.plot(p_ext, label="Price of the market")
plt.plot(a_ext, label="Asks")
plt.plot(b_ext, label="Bids")
plt.plot(mid_price_ext, label="Mid-price")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.title("Price evolution with aggressive and passive trader behavior")
plt.show()

```

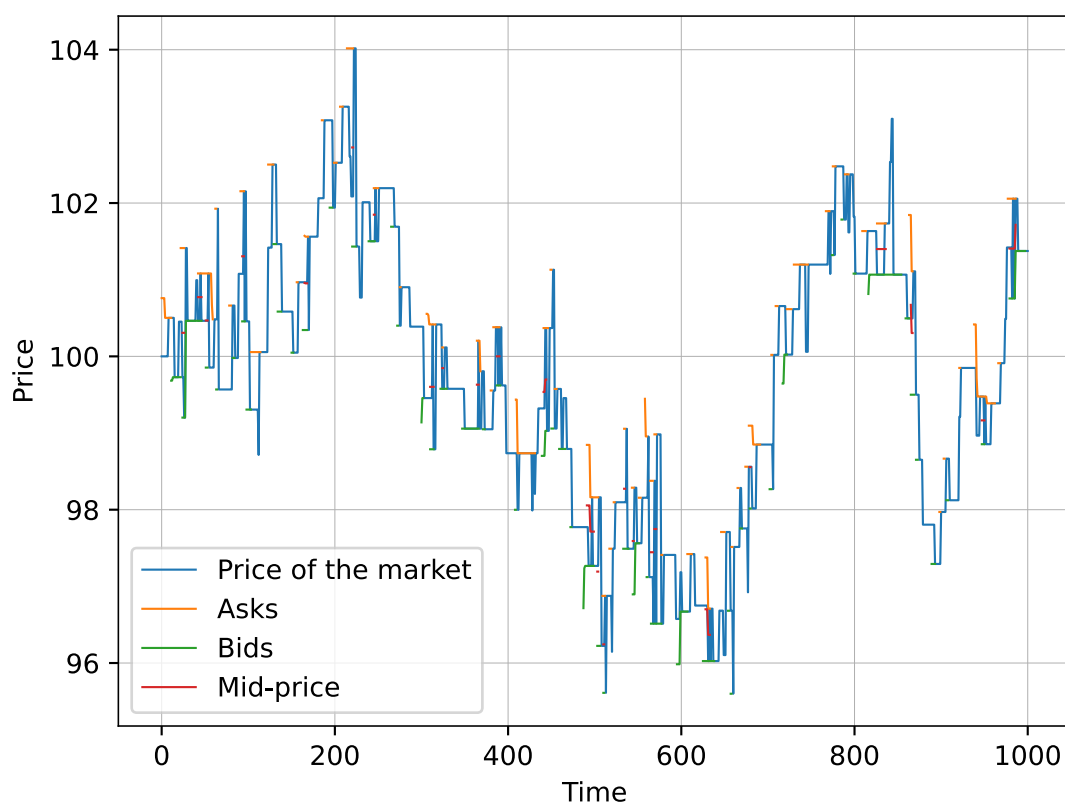


Figure 10: Price evolution with aggressive and passive trader behavior

2. Record the time series of the market price and compute the returns.

```
returns_ext = returns(p_ext)

plt.figure()
plt.plot(returns_ext)
plt.title("Time series of returns of the market price (extended model)")
plt.xlabel("Time")
plt.ylabel("Returns")
plt.show()
```

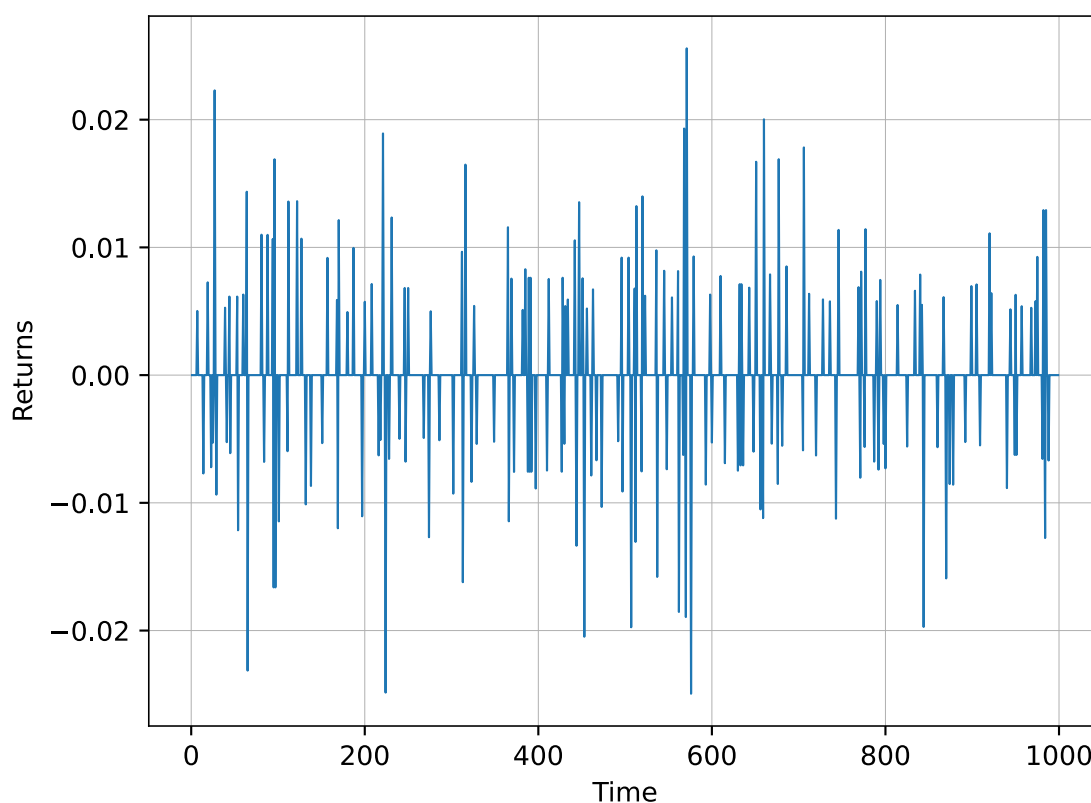


Figure 11: Time series of returns of the market price (extended model)

3. Plot the histogram of returns and calculate basic statistics (mean, variance).

```
plt.figure()
plt.hist(returns_ext, bins=25)
plt.title("Histogram of Returns (extended model)")
plt.xlabel("Returns")
plt.ylabel("Frequency")
plt.show()

print("Mean of returns:", np.nanmean(returns_ext))
print("Variance of returns:", np.nanvar(returns_ext))
```

```
Mean of returns: 2.359222362809064e-05
Variance of returns: 1.9842517325582804e-05
```

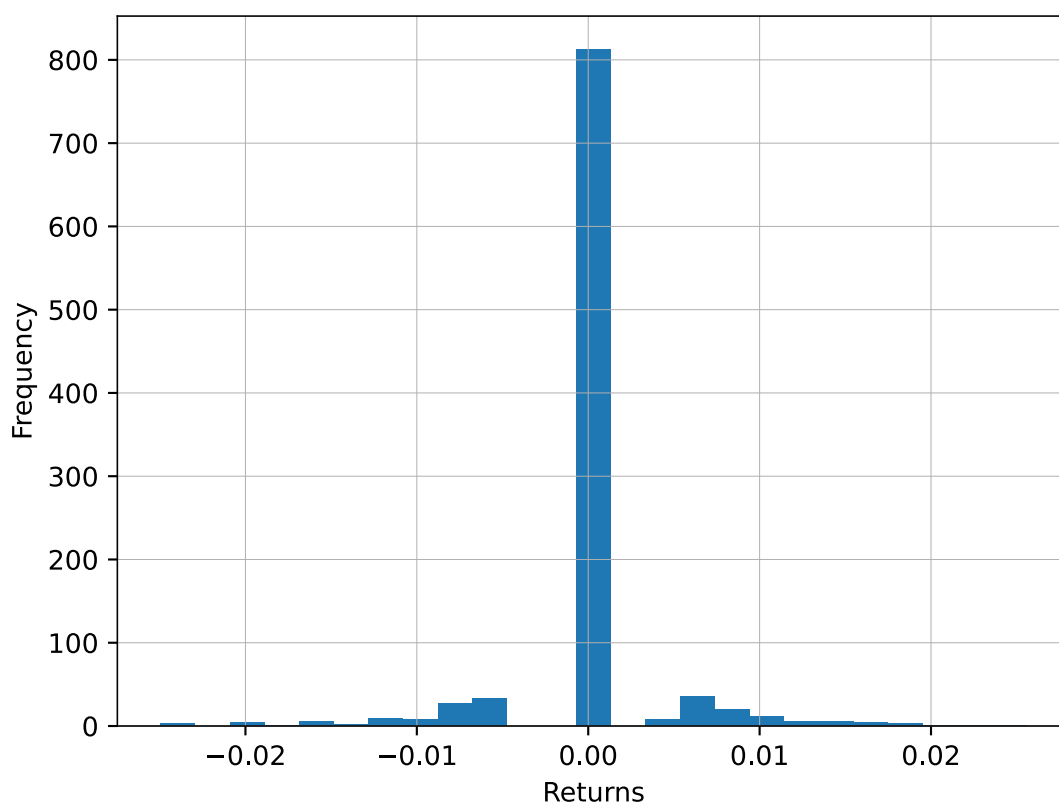


Figure 12: Histogram of Returns (extended model)

4. Compute and plot the autocorrelation function (ACF) of the return series.

```
plot_acf(returns_ext, lags=np.arange(51))
plt.title("ACF of returns (extended model)")
plt.xlabel("Lags")
plt.ylabel("Autocorrelation")
plt.show()
```

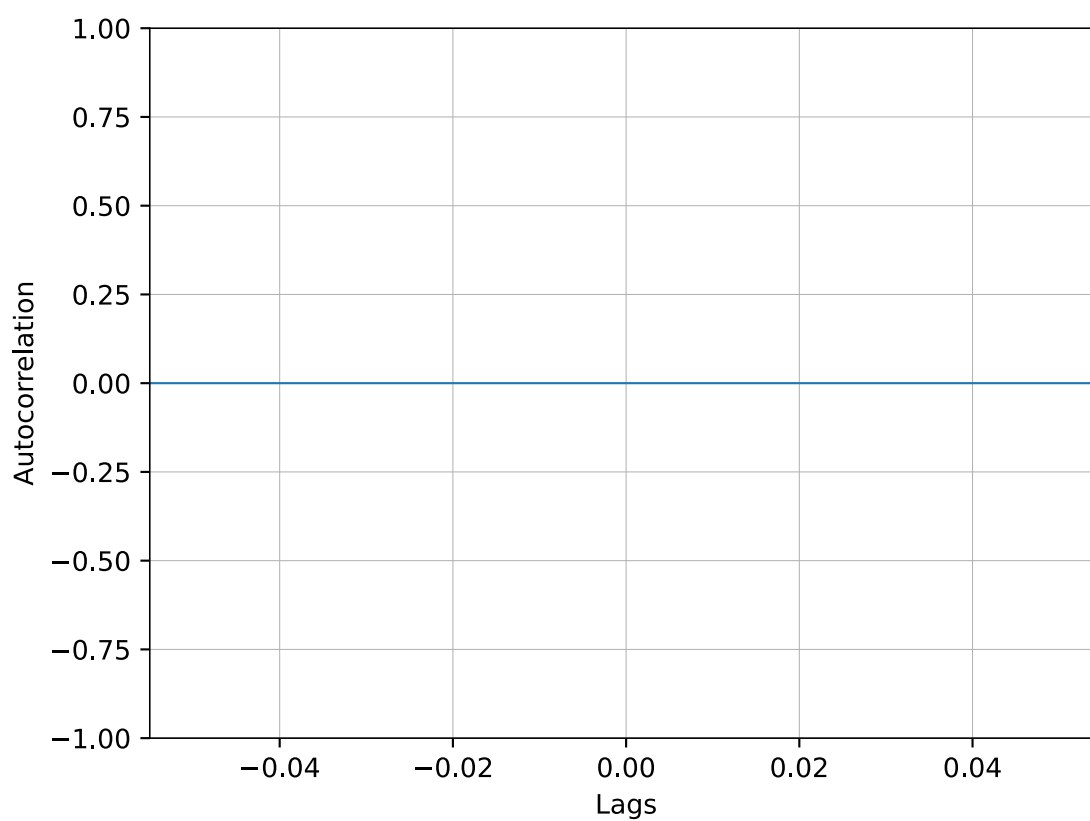


Figure 13: ACF of returns (extended model)

5. Discuss how the introduction of heterogeneous order sizes and trader behavior affects the price dynamics compared to the baseline model. Comment on any observed changes in volatility clustering or the fat-tailed nature of the return distribution.